6. Estructuras de repetición – Conteo de iteraciones

En esta sección se aborda exclusivamente el cálculo de cuántas veces se ejecuta un ciclo, sin introducir nuevas estructuras ni repetir definiciones ya vistas.

El objetivo es que el estudiante pueda analizar el código y determinar la cantidad de iteraciones, tal como se solicita en el cuestionario.

6.1. Cantidad de iteraciones en un ciclo for con range()

Cuando se utiliza un ciclo for con range(), la cantidad de repeticiones depende de los valores indicados en el rango.

range(n) → el ciclo se ejecuta n veces.

range(inicio, fin) → se ejecuta desde inicio hasta fin - 1.

range(inicio, fin, paso) → el ciclo avanza de acuerdo al valor de paso.

Ejemplo conceptual:

range(5) → 5 repeticiones

range(2, 10, 2) → valores: 2, 4, 6, 8 → 4 repeticiones

El valor final nunca se incluye.

Ejemplo:

```
for i in range(5):
    print(i)
```

Salida por pantalla

```
0
1
2
3
4
```

6.2. Cantidad de iteraciones en un ciclo while

En un ciclo while, la cantidad de repeticiones depende de:

El valor inicial de la variable de control.

La condición lógica del ciclo.

El cambio que sufre la variable en cada iteración.

Para determinar cuántas veces se ejecuta el ciclo, es necesario seguir el valor de la variable paso a paso hasta que la condición deje de cumplirse.

Ejemplo conceptual:

Si una variable comienza en 0 y se incrementa hasta que deja de cumplir una condición < 3, el ciclo se ejecuta 3 veces.

```
contador = 0

while contador < 3:
    print(contador)
    contador += 1
```

Salida por pantalla

```
0
1
2
```

Si una variable comienza en 10 y se decrementa de 2 en 2 mientras sea mayor que 0, el ciclo se ejecuta 5 veces.

```
contador = 10

while contador > 0:
    print(contador)
    contador -= 2
```

Salida por pantalla

```
10
8
6
4
2
```

6.3. Ciclos for con salto (paso distinto de 1)

Cuando el ciclo for utiliza un paso distinto de 1:

El número de iteraciones se reduce.

Es necesario listar mentalmente los valores generados por el range().

Esto permite determinar con precisión cuántas veces se ejecuta el bloque del ciclo.

6.4. Ciclos combinados

Son estructuras en las que un ciclo está **dentro** de otro (anidamiento de bucles) o donde se combinan diferentes tipos de ciclos (**for** con **while,** etc.).

```
for x in range(3):
    for y in range(2):
        print(f"x = {x}, y = {y}")
```

```
x = 0, y = 0
x = 0, y = 1
x = 1, y = 0
x = 1, y = 1
x = 2, y = 0
x = 2, y = 1
```

Cada vuelta de x contiene dentro 2 vueltas de y. De esta forma, cada vuelta del ciclo de x tiene un ciclo de vueltas de y.

- ## Ejemplo más complejo

Se necesita un programa para cargar las ventas ordenadas por mes del año 2024. El ingreso de ventas cambia de mes cuando se ingresa 0. La cantidad de ingresos de ventas no son las mismas en cada mes. Al final del programa se debe mostrar el total de las ventas de todos los meses

Análisis del enunciado

ordenadas por mes del año 2024: sabemos que todos los años tienen la misma cantidad de meses por lo que sabemos de antemano la cantidad de vueltas que dará el ciclo externo (12), ciclo for.

La cantidad de ingresos de ventas no son las mismas en cada mes: No sabemos cuántas ventas hubo por mes, entonces la cantidad de vueltas que dé el ciclo interno dependerá del usuario y la cantidad de ventas que deba cargar, ciclo while.

El ingreso de ventas cambia de mes cuando se ingresa 0: El fin de carga de ventas por mes está regido por el 0, es decir que cuando termino de cargar ventas de un mes debo ingresar 0 para que cambie el mes.

Al final del programa se debe mostrar el total de las ventas de todos los meses: Cuando todas las vueltas de ambos ciclos finalizan, el programa debe mostrar lo recaudado en total de todos los meses. Es decir que se necesita acumular todas las ventas.

```
venta = 0
acumulador_ventas = 0

for x in range (12):
    print("Mes:", x + 1)
    venta = int(input("Ingrese importe de venta: "))
    while venta != 0:
        acumulador_ventas += venta
        venta = int(input("Ingrese importe de venta"))

print(f"El total de ventas en el año es: {acumulador_ventas}")
```

- Se inician las variables de venta y acumulador_ventas en 0.

- Un ciclo for de 12 vueltas, uno por cada mes, indica en que mes estamos.
- Dentro del ciclo for un ciclo while que pide ingresar ventas.
- Una variable acumuladora se encarga de sumar en cada vuelta el importe de la venta que se ingresa.
- Al final del programa se muestra el total acumulado durante el año.